

Marketing AI Agent — Dashboard Guide

What the dashboard is, every library it uses and why, and how the code works.

Location: `eval/dashboard/` · Branch: `crm`

1. What the dashboard is

A web UI for the Marketing AI Agent. After signing in, a user sees three pages:

- **Dashboard** — start/pause the two pipelines (Email Outreach, Blog Publishing), see lead statistics, a leads-by-source bar, and a filterable table of 248 leads.
- **History** — two tables: emails sent and replies received.
- **Blogs** — blog tracker with Posted / Not Posted counts.

Clicking any table row opens one shared side **drawer** with the full details (lead, email, reply or blog), with Previous/Next buttons and links between related items (e.g. lead → its email conversation → the reply).

Important: the data and login are currently **mock**. `core/data.js` holds dummy data and `core/auth.js` accepts any valid-looking email with an 8+ character password. Both are written so a real backend (REST API / Supabase) can replace them by changing only those two files.

The dashboard was migrated from plain HTML/JS (`index.html` , `js/*.js`) to **Next.js**. The old files are deleted in git status; `styles.css` became `app/globals.css` , and `js/api.js` , `js/data.js` moved to `core/` .

2. Libraries and tools — what and why

The project has exactly **three** npm dependencies (`package.json`). There are no dev-dependencies.

Library	Version	What it is	Why we use it
next	<code>^15.5.0</code>	React framework (routing, build, dev server).	Gives file-based routing (folders in <code>app/</code> become URLs), route groups and shared layouts, <code>redirect()</code> , per-page metadata (tab titles), a dev server with hot reload and an optimized production build — without hand-writing a router or bundler config.
react	<code>^19.0.0</code>	UI library.	Components, state (<code>useState</code>), side-effects (<code>useEffect</code>), context, refs. It replaced the old manual DOM building, so the UI re-renders automatically when state changes.
react-dom	<code>^19.0.0</code>	Renderer that puts React into the browser DOM.	Required companion of React for web apps; Next.js uses it internally for rendering and hydration.

Parts of Next.js that are used (no extra install)

- `next/link` — `Link` for client-side tab navigation without full page reloads.

- `next/navigation` — `useRouter` (push/replace), `usePathname` (which tab is active), `redirect` (server redirect from `/` to `/dashboard`).
- `metadata` export — sets the browser tab title ("*Blogs · Marketing AI Agent*").
- `next.config.mjs` — enables `reactStrictMode`, which double-runs effects in development to expose bugs early.
- `jsconfig.json` — the alias `@/*` → project root, so imports read `@/core/api` instead of `../../core/api`.

External resources and browser APIs

Item	Used for
Google Fonts: Inter (400–800) and JetBrains Mono (400, 500)	Inter is the UI font; JetBrains Mono is used for IDs (<code>.mono</code> class). Loaded by <code><link></code> tags in <code>app/layout.js</code> with <code>preconnect</code> for speed.
<code>localStorage</code>	Remembers the chosen light/dark theme across visits.
<code>sessionStorage</code>	Holds the mock login session (<code>{email}</code>); cleared when the tab closes.
<code>matchMedia</code>	Detects the OS dark-mode preference when no theme is saved.
CSS custom properties	All colors are variables in <code>:root</code> ; dark mode overrides them under <code>[data-theme="dark"]</code> .
Inline SVG	All icons are drawn by hand in <code>Icon.js</code> .

What we deliberately do NOT use (and why)

- **No UI kit** (Tailwind, MUI, shadcn) — one 279-line stylesheet covers everything and avoids a dependency and build step.
- **No icon library** (lucide, react-icons) — only ~25 icons are needed; inline SVG keeps the bundle small.
- **No state library** (Redux, Zustand, React Query) — React Context in `DataProvider` is enough for six data sections.
- **No markdown library** — blog/email bodies need only headings, lists, quotes and links, so `FormattedContent.js` does it in ~30 lines.
- **No chart library** — the only chart is a single stacked bar made from flex-grow `<span>` s.
- **No auth/HTTP library yet** — mock for now. Planned: `@supabase/supabase-js` for auth (named in comments in `auth.js`) and plain `fetch()` for the API.

3. Folder structure

```
eval/dashboard/
├ package.json, next.config.mjs, jsconfig.json, .gitignore
├ app/                                ← Next.js routes
│ ├── layout.js                       root HTML, fonts, theme script
│ ├── globals.css                     all styling (light + dark)
│ ├── page.js                         "/" → redirects to /dashboard
│ ├── login/page.js                  "/login"
│ └── (app)/                          route group: pages that need sign-in
│     ├── layout.js                  wraps pages in AppShell
│     ├── dashboard/page.js          "/dashboard"
│     ├── history/page.js            "/history"
│     └── blogs/page.js              "/blogs"
├ components/                        ← React components
│ AppShell, DataProvider, DrawerProvider, drawerKinds,
│ DashboardView, HistoryView, BlogsView, LoginView,
│ ui, Icon, OpenRow, FormattedContent
└ core/                              ← non-UI logic
    api.js, data.js, auth.js, format.js
```

The folder `(app)` in parentheses is a **route group**: it does not appear in the URL but lets the three signed-in pages share one layout (`AppShell`) while `/login` stays outside it.

4. How a request flows

1. User opens `/` → `app/page.js` redirects to `/dashboard`.
2. `app/layout.js` renders the HTML shell; an inline script sets `data-theme` before first paint (no light/dark flash).
3. `(app)/layout.js` renders `AppShell`. It checks `auth.isAuthenticated()`; if not signed in it redirects to `/login`, otherwise it renders `DataProvider` → `DrawerProvider` → `Topbar` + `page`.
4. The page component (e.g. `DashboardView`) calls `useSection('leads')`; data is fetched once, on first use, through `core/api.js`.
5. Clicking a row calls `open('lead', id)` on the drawer, which loads the detail and shows it.

5. Code explained, file by file

app/layout.js — root layout

Imports `globals.css` (global styles), exports `metadata` with a title template, and defines `RootLayout`. In `<head>` it loads the Google Fonts and injects `THEME_SCRIPT`: a tiny string of JS that reads `localStorage.theme` (or falls back to the OS preference) and sets `document.documentElement.dataset.theme`. It runs before React, so the correct theme is painted immediately. `suppressHydrationWarning` on `<html>` stops React complaining that the attribute differs between server and browser.

app/page.js, app/login/page.js, app/(app)/*

Thin route files. Each one just renders a view component (`LoginView` , `DashboardView` , ...) and sets its page title. All real logic sits in `components/` so route files stay trivial.

core/auth.js — authentication (mock)

- `signIn(email, password)` — waits 600 ms (simulates network), validates email by regex and password length ≥ 8 , stores `{email}` in `sessionStorage` .
- `getSession()` / `isAuthenticated()` — read the session; wrapped in try/catch because storage can be blocked.
- `signOut()` , `requestPasswordReset(email)` — clear the session / validate the email.
- Each function has a comment naming the Supabase call that will replace it (`supabase.auth.signInWithPassword` etc.). No other file knows how auth works.

Security note: because the session lives in `sessionStorage` and is checked in the browser, this guard only controls the UI. Real protection must come from the backend verifying a token on every API call.

core/data.js — dummy data

Exports `pipelines` , `leads` (248), `sentEmails` , `replies` and `blogs` . A seeded random generator (`rng` , a mulberry32-style function) plus `shuffle` makes the data **deterministic**: every reload gives the same numbers, and the totals always add up (248 = 144 LinkedIn + 60 X + 44 Other = 152 Responded + 96 Awaiting). It is only imported by `api.js` .

core/api.js — the data-access layer

Every function mimics an HTTP endpoint (the endpoint is written in the comment above it): `getPipelines` (GET /pipeline), `startPipeline` / `pausePipeline` (POST), `getLeads` , `getLeadStats` , `getLead` , `getSentEmails` , `getSentEmail` , `getReplies` , `getReply` , `getBlogs` , `getBlog` .

- `wait()` adds fake latency so loading skeletons are visible.
- `clone()` (JSON round-trip) returns copies so UI code can never mutate the source data.
- `omit()` removes heavy fields from list responses (e.g. email `body` , blog `content`) — list endpoints return summaries, detail endpoints return everything. This mirrors how a real API should behave.
- `find()` throws `Not found` , which the UI turns into an error box with a retry button.
- `getReply` attaches `originalEmail` so the drawer can link back to the email being answered.

Going live: replace each body with `fetch()` ; no component changes needed.

core/format.js — shared helpers

- `pad(n)` → 7 becomes "007" for item numbers.
- `fmtDate(iso, mode)` — formats dates in three styles (`full` , `table` , `date`) using `toLocaleDateString` with `timeZone: 'UTC'` , so server and browser render identical text (avoids hydration mismatches).
- `toneOf(text)` — maps a status word to a badge color (Running/Delivered/Replied/Posted/Responded → green; Paused/Awaiting/Not Posted → amber;

Failed → red; else gray).

- `SOURCES` — label, icon and color for LinkedIn, X and Other; used by tables, the legend and the stacked bar.

components/AppShell.js — signed-in frame

Client component (`'use client'` — needed for hooks and browser APIs). `Topbar` renders tab links (active tab detected with `usePathname`, marked with `aria-current`), a theme toggle (flips `data-theme` and saves to `localStorage`) and Sign out. `AppShell` runs the auth guard inside `useEffect`: `authenticated` → `ready=true`; otherwise `router.replace('/login')`. It renders nothing until ready, preventing a flash of protected content.

components/DataProvider.js — shared data store

A React Context holding six sections: `pipelines`, `stats`, `leads`, `sent`, `replies`, `blogs`. Each is `{status, data}` where status is `idle | loading | ready | error`.

- `FETCHERS` maps each section to its `api` function.
- `load(key)` sets loading → awaits the fetch → sets ready or error. A `busy` ref (a Set) prevents duplicate parallel requests.
- `ensure(key)` loads a section only the **first** time it is needed (tracked in `started`); a failed section waits for the user to click retry.
- `update(key, fn)` modifies cached data locally (used for optimistic pipeline updates).
- `useSection(key)` — the hook pages use: triggers `ensure` in an effect and returns `{status, data, reload}`.
- `filter` (status + source) also lives here, so filters and loaded data survive switching tabs. `visibleLeads()` applies the filter and is shared by the table and the drawer's Previous/Next.
- The context value is wrapped in `useMemo` and callbacks in `useCallback` to avoid needless re-renders.

components/DrawerProvider.js — the single drawer

One drawer serves leads, emails, replies and blogs. State `s = {open, kind, id, status, detail}`.

- `open(kind, id)` — remembers the previously focused element, sets loading, fetches detail via the kind's `load`. A `token` counter ignores stale responses if the user clicks Next quickly (race-condition guard).
- `close()` — hides the drawer and returns keyboard focus to the element that opened it.
- `goto(page, kind, id)` — if already on that page, just opens; otherwise stores the target in `pending` and calls `router.push`. A `useEffect` on `pathname` opens the pending item after navigation, or closes the drawer if there is none.
- `step(delta)` — Previous/Next through the current list (`kinds[kind].list()`).
- **Accessibility:** `role="dialog"`, `aria-modal`, `inert` when closed, Escape closes, Tab/Shift+Tab are trapped inside the drawer, and focus is restored when content changes.
- Shows skeletons while loading, `ErrorBox` with retry on failure, and scrolls the body to top on each change.
- Uses refs (`kindsRef`, `gotoRef`, `ensureRef`) so `open` stays stable without stale closures.

components/drawerKinds.js — what each drawer shows

`getKinds({st, filter}, goto)` returns four descriptors: `lead`, `sent`, `reply`, `blog`. Each has `section` (backing data), `heading`, `list()`, `load(id)` and `describe(detail)` which returns the title, badge, meta rows, action button and body text. Adding a new drawer type means adding one object here — the drawer itself does not change. Examples: a lead's action opens its email conversation; a sent email links to its reply; a reply links back to the original email; a posted blog links to the live post.

components/DashboardView.js — page 1

- **PipelineControl** — a card per pipeline plus Start All / Pause All. `act()` uses an **optimistic update**: the badge flips immediately; if the API call fails it rolls back to the saved previous statuses and shows an error notice.
- **StatCards** — Total, Awaiting, Responded from `getLeadStats`, each with skeleton / retry / value states.
- **SourceBreakdown** — a stacked bar built from spans with `flexGrow` equal to each source's count, plus a legend; has an `aria-label` summary for screen readers.
- **LeadsTable** — filter `Chip`s (status and source, with `aria-pressed`) and a `DataTable` of rows wrapped in `OpenRow`.

components/HistoryView.js and BlogsView.js — pages 2 and 3

`HistoryView` reuses one `HistoryCard` component twice (Emails Sent / Replies Received), passing only different columns and row renderers. `BlogsView` computes Posted vs Not Posted counts and renders the blog table. Both follow the same pattern: `useSection` → `DataTable` → `OpenRow`.

components/LoginView.js — login page

Two modes in one form — `signin` and `reset` — driven by a `COPY` object for the text. Uses uncontrolled inputs through refs, a `busy` flag to block double-submit, and a message area with `role="alert"`. Signed-in users are redirected away. It adds `auth-view` to `<body>` so CSS hides the top bar and centres the card (class removed on unmount). Includes a show/hide-password toggle and moves focus to the failing field on error.

components/ui.js — small shared pieces

`Badge`, `StatusBadge`, `SourceCell`, `SiteCell`, `Skeleton` (loading placeholder), `ErrorBox`, `EmptyBox`, `CardHead`, `MetaRow`, `ExtLink` (opens in new tab with `rel="noopener noreferrer"`), and `DataTable`, which picks the right output for a section: skeleton while idle/loading, error box on failure, empty message when no rows, otherwise the table.

components/OpenRow.js, Icon.js, FormattedContent.js

- **OpenRow** — a `<tr>` that is keyboard-focusable (`tabIndex=0`), opens the drawer on click or Enter, and highlights when selected.
- **Icon** — dictionary of SVG path strings. Two kinds: `stroke` icons (shared attributes, colored by `currentColor` so they follow the theme) and `logo` icons (LinkedIn, X, WordPress, Medium, Dev.to with own colors). Rendered with `dangerouslySetInnerHTML`; this is safe only because the strings are static and trusted.

- **FormattedContent** — turns plain text into React elements: splits on blank lines, then detects `-` lists, `1.` lists, `>` quotes, `##` / `###` headings and paragraphs; `linkify()` uses a regex to auto-link URLs and emails. Because it builds React elements (not HTML strings), content cannot inject scripts.

app/globals.css — styling

One file, ~279 lines. Light theme values live in `:root` as variables (`--bg`, `--card`, `--accent`, `--green`, `--amber...`); the dark theme overrides them in `[data-theme="dark"]`, so components never need theme-specific code. Layout uses CSS Grid (`grid2`, `grid3`) and Flexbox; `.skel` has a shimmer animation; `.drawer` / `.overlay` slide in using an `.open` class; `:focus-visible` outlines support keyboard users.

6. Key design ideas

Idea	Benefit
All data access in <code>core/api.js</code>	Swap mock for a real backend in one file.
All auth in <code>core/auth.js</code>	Swap mock for Supabase in one file.
Sections with idle/loading/ready/error	Every widget handles loading, failure and retry consistently.
Lazy loading via <code>ensure</code>	Only fetch what the visitor actually opens.
Single drawer driven by <code>drawerKinds</code>	One implementation, four item types, easy to extend.
Optimistic updates with rollback	Instant-feeling buttons that stay correct on failure.
CSS variables + early theme script	Flicker-free dark mode with no duplicated styles.
Accessibility built in	Focus trap, Escape, ARIA roles, keyboard-openable rows.

7. Running it

```
cd eval/dashboard
npm install
npm run dev      # http://localhost:3000 (hot reload)
npm run build    # production build
npm start        # serve the production build
```

Any email like `you@company.com` with a password of 8+ characters signs in (mock).

8. Next steps to go live

- Replace `core/api.js` function bodies with `fetch()` calls to the listed endpoints.
- Replace `core/auth.js` with Supabase (`npm i @supabase/supabase-js`) and verify tokens server-side.
- Delete `core/data.js` once nothing needs dummy data.
- Consider adding pagination to the leads table once real data exceeds a few hundred rows.